Foundations of Artificial Intelligence - 2026.1

Sequential Decisions under Uncertainty - Competitive Environments

Felix Mohr

Universidad de La Sabana

Chía, April 28, 2026

Decision Making Situations

Decision Situation Matrix

Horizon	Action Effects: Certainty	Risk	Uncertainty
Atomic	✓	✓	✓
Sequential	✓	✓	NOW (33%)

Cooperative vs Competitive Situations

Suppose that we have k agents in a *cooperative* scenario.

In such a scenario, we can pretend that in each time step t there will be actions $A_t^1, .., A_t^k$, which are coordinates by the group.

At least if we assume that every agent might follow an optimal group decision, we could simply treat this as a standard MDP.

Hence, we are more interested in a competitive scenario, where agents do not coordinate their actions.

Class Overview

1. A Ceteris Paribus Approach to Competitive Multi-Agent Situations
2. Alternating Markov Games
3. Exploration Based On Winning Probabilities

Class Overview

1. A Ceteris Paribus Approach to Competitive Multi-Agent Situations
2. Alternating Markov Games
3. Exploration Based On Winning Probabilities

Reinforcement Learning for Competitive Multi-Agent Situations

The Transitivity Assumption

Suppose we have a game with $k \geq 2$ players, and policies $\pi_2, .., \pi_k$ are available.

From the viewpoint of each player 1, the policies $\pi_{\neq 1}$ induce an MDP.

Possible Task: Learn an optimal policy π_1^* for this MDP.

But then what happens if, in the same game, we play against other policies $\pi_2', .., \pi_k'$?

More reasonable task: Learn a policy π_1^* that is optimal in *all* possible MDPs.

Reinforcement Learning for Competitive Multi-Agent Situations

The Transitivity Assumption

It is entirely unclear whether such a policy π_1^* can even exist.

This also depends on the definition of utilities: If utility depends on efficiency, it is usually not possible.

We will here assume that

- player 1 starts in a random initial state s_0
- rewards for player 1 only at the end, typically in $\{-1, 0, 1\}$
- $\gamma = 1$
- there is a known upper bound T on the maximum number of any trial

Reinforcement Learning for Competitive Multi-Agent Situations

Since $\gamma = 1$, only the final result counts (but not how fast we win/lose).

Under these assumptions, it is reasonable that a generally optimal policy may exist.

Intuition: If A is a stronger opponent than B, then the ability of winning against player A should also imply the ability of winning against B.

We should learn to play an MDP induced by the strongest possible opponent players.

Usually we do not have such strong opponent policies available to generate trials.

Reinforcement Learning for Competitive Multi-Agent Situations

If the assumption holds for all players, we can use a cross-MDP policy iteration idea:

Initialize $\pi_1, .., \pi_k$ randomly. Then, in an endless loop run

- Fix current player $i \leftarrow t \ \% \ k$
- Find π_i^* for the current MDP induced by $\pi_{\neq i}$
- Update $\pi_i \leftarrow \pi_i^*$
- If for k turns no policy has changed, stop

A fix point here would imply that we have found stable policies $\pi_1^*, .., \pi_k^*$.

Reinforcement Learning for Competitive Multi-Agent Situations

The above approach has several disadvantages:

1. Sample Efficiency

   For every game played in this way, we only update the knowledge of one player

2. Learning Efficiency

   Learning optimal policies for overly simplistic opponents is wasted time, because this learned knowledge is destroyed as soon as opponents are updated.

This approach would be typically *very* slow and learn a lot of irrelevant stuff ...

Class Overview

1. A Ceteris Paribus Approach to Competitive Multi-Agent Situations
2. Alternating Markov Games
3. Exploration Based On Winning Probabilities

Alternating Markov Games

An Alternating Markov Game is described as

$$\mathcal{G} = \langle S, \{A_i\}_{i=1}^k, \mathbb{P}, \{r_i\}_{i=1}^k, \gamma, \eta \rangle$$

where everything is as in a standard MDP except that

- A_i is the action space of player i,
- $r_i : S \to \mathbb{R}$ is the reward function of player i,
- $\eta : S \to \{1, \dots, k\}$ determines the *active player* in each state.

At each step t:

1. The active player $i = \eta(s_t)$ selects an action $a_t \in A_i$ according to $\pi_i(a_t \mid s_t)$.
2. The environment transitions to $s_{t+1} \sim P(\cdot \mid s_t, a_t)$.
3. All players receive rewards $r_j(s_t)$.

Alternating Markov Games

Reinforcement Learning would now happen for all players simultaneously.

In an infinite loop, we would

- create a trial τ based on all players' current policies.
- propagate the projected trial τ_i with player i's rewards in all states to each player
- each player performs a standard RL and PImp step

The trial for player i should

- only contain states $s_1^i, s_2^i, ..$ with $\eta(s_t^i) = i$, and
- as reward $r(s_t^i) := \sum_{s \in \tau[s_{t-1}^i : s_t^i]} r_i(s)$, thanks to $\gamma = 1$; rewards are now stochastic

Alternating Markov Games

Since we want to share knowledge across k GPI processes that would usually be generated in k independent PE steps, it is good to think of an alternative architecture.

One little invasive approach:

Make the PE routine a function of a trial τ (instead of conducting trials itself).
The PImp routine can stay the same.

Reinforcement Learning for 2-Player Games

Alternating Markov Games

The most important change induced by Alternating Markov Games is that we can *synchronize* the learning processes of all players.

This has at least two effects

- Better usage of compute resources invested in trials:
  - Before: For 1 learning step of k agents we needed k trials.
  - Now: For 1 learning step of k agents we need 1 (shared) trial.
- Trials may implement updates in the opponents immediately:
  - Before: some number of trials for same opponent
  - Now: each trial can be based on updated opponents

Reinforcement Learning for 2-Player Games

Alternating Markov Games

There are special cases in which we can proceed slightly more efficient.

Consider a zero-sum game in which the only reward is in the last action.

In this case, we would learn a single q-function for both players simultaneously.

One can think of this literally as having a single agent that simultaneously plays both players.

Then we are (almost) back in the standard MDP case (but now with a state for each player).

Reinforcement Learning for 2-Player Games

Alternating Markov Games

The only technical difficulty is that the reward at the end is only for one of the two players:

If it is 1 for player 1, the reward is -1 from the perspective of player 2.

We somehow must take this into account when updating q-values.

The simplest way to do this is to multiply the previous iteration's utility with $-\gamma$ instead of γ, typically with -1.

Thereby, the bipolar aspect of the game is reflected in the q-updates.

Class Overview

1. A Ceteris Paribus Approach to Competitive Multi-Agent Situations
2. Alternating Markov Games
3. Exploration Based On Winning Probabilities

Exploration Based On Winning Probabilities

Shared trials do not use Exploring Starts anymore.

This implies that we need other ways to estimate $\hat{q}(s, a)$ for $a \neq \pi_i(s)$.

But even if we wouldn't use centrally generated trials, Exploring Starts would waste a large portion of the resources learning such irrelevant q-values:

There is no point in learning *all* $q(s, a)$ values if most regions of the MDP are bad.

Then we must use our own policy for exploration.

Exploration Based On Winning Probabilities

How can we make sure that our policy increases the probability to visit *relevant* states?

A state is relevant if it significantly influences $v(s_0)$ of an optimal policy.

So we might want to ignore states for which it is almost sure that they will not be visited by an optimal policy.

Unfortunately, we do not know this before.

Exploration Based On Winning Probabilities

In a Bernoulli Game ($r \in \{0, 1\}$, once per trial, $\gamma = 1$), each $v^\pi(s)$ is the probability to win after s under π.

An optimal policy will seek to avoid states with low $v(s)$.

This gives a natural motivation to explore in such a way: Sample actions with a probability that is proportional to the current belief to win the game with that action.

$$\pi(a|s) = \frac{\hat{q}(s, a)}{\sum_{a'} \hat{q}(s, a')}$$

Exploration Based On Winning Probabilities

This method has two problems:

1. it does not take into account our certainty about the estimates and
2. it might not converge to a policy with (almost) probability 1 for a specific action.

Both issues can be solved by picking actions that maximize the UCB score instead:

$$u(s,a) = \hat{q}(s,a) + \sqrt{\frac{\log \sum_{a'} N[s,a']}{N[s,a]}}$$

This is not randomly picking yet exploring forever while converging.

Exploration Based On Winning Probabilities

Suppose that trials are now generated with this randomly exploring policy, say π'.

The problem is now that PE does not evaluate π anymore but π'.

This can break the assumption that PE makes $\hat{q}^{\pi}$ converge to q^{π} (because we get $\hat{q}^{\pi'}$).

However, if π greedily follows $\arg\max_a \hat{q}(s, a)$ and π' follows $\arg\max_a u(s, a)$, then these policies are almost always identical for sufficiently large sample sizes.

Summary

Main takeaways:

- Competitive Multi-Agent Environments require a special treatment
- A rather efficient approach is to generate shared trials and synchronously update policies across players
- In this case, we cannot use Exploring Starts anymore but can control exploration via the policy.
- Even though we now have an exploring policy, $\hat{q}$ converges to the q in the maximally competitive scenario.